开源项目分享

60 美元、103 小时，做出一款上架 App Store 的 iOS 应用

门槛低，上限高 —— 来自非开发人员的 AI Coding 实践

案例：份子钱银行

业余时间约 3 个月

Cursor 协作约 103 小时

分享人：fingalyang

AI 生成

什么是 AI Coding

定义

用自然语言描述需求，由 AI 生成、修改和调试代码的开发方式

分工

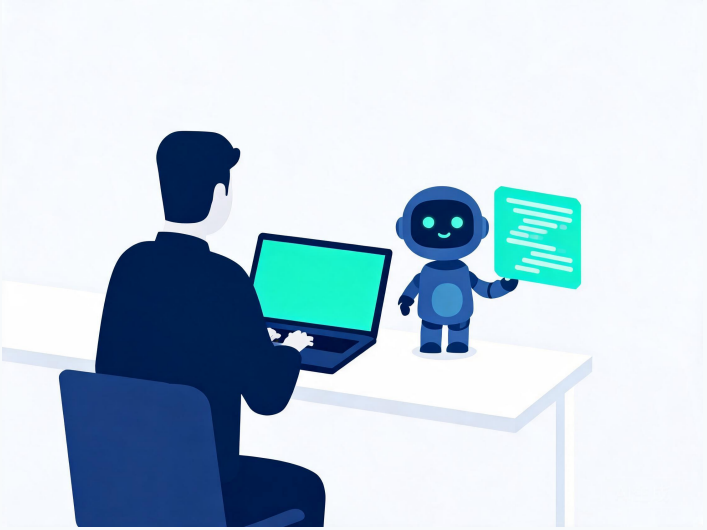
人负责定目标、做取舍、管验收；AI 负责具体的代码实现

方式

在编辑器里边对话边开发：人提需求，AI 写代码，人来验收

实践

本次分享的 iOS 应用，全部代码均由人与 Cursor 协作完成



AI 写代码，人做判断

Cursor：你的 AI 编程搭档

是什么

基于 VS Code 深度定制的 AI 原生代码编辑器

能做什么

对话生成代码、跨文件改写、自动排错，还能按需配置开发环境

怎么协作

人用自然语言下指令，Cursor 读写工程文件，人来审查确认

本次实践

从创建工程到提交上架，全程约 103 小时与 Cursor 协作完成



在 Cursor 里，对话就是开发

CONTENTS · 分享路径

七个模块，从认知到落地

01	开场	先认识 AI Coding 与 Cursor
02	背景与成果	从零经验到上架 App Store
03	常见问题	钱、时间与适用范围
04	核心原则	100% AI Coding 的人机分工
05	方法 · 技巧 · 经验	协作方法、四个技巧与八条实战经验
06	实操时间线	三个月里的每一轮循环
07	量化分析 · 答疑	工时、返工率、缺陷数据与自由交流

为什么做这次实践

背景

网络工程专业（2013 年毕业），长期从事信息化建设与网络信息安全管理，持有 CISSP / CISP / CDGA 证书

语言

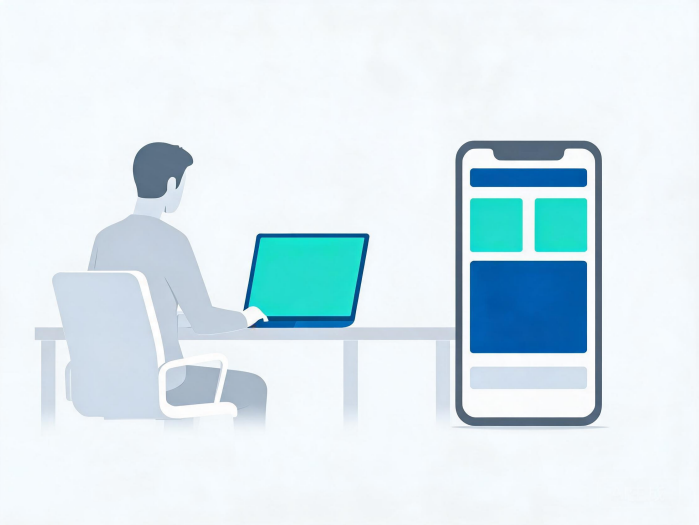
熟悉 Python、C、JavaScript、PHP——但没有大型软件开发经验，也没有移动应用开发经验

路径

先完成瑞通 AI Coding 课程，再立即动手验证；实践选择 iOS 应用开发

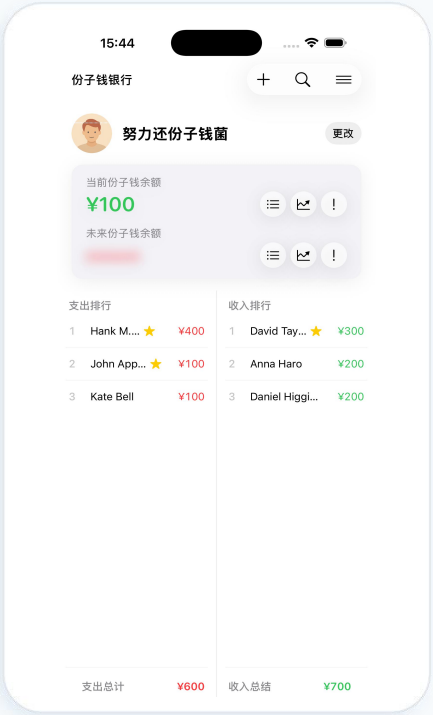
环境

macOS + Xcode + Cursor；具体语言与框架不事先指定，由 Cursor 按需配置



先学会与 AI 协作，再让 AI 干活

案例 App：「份子钱银行」



一款本地记账应用，记录每一份人情往来，让随礼与回礼一目了然。

100%

全部代码由人与 Cursor 协作完成

约 3 个月

利用业余时间，从零经验到正式上架

31 次提交

每轮按“方案—确认—开发—验收”推进

持续迭代

发布后推进 v1.0.2、软著与推广

核心结论：人负责目标、取舍和验收，AI 负责执行。

三个最常被问到的问题



Q1 需要花很多钱吗?

不需要。两个月的模型费用共 60 美元，使用最强思考强度和 fast 模式，业余开发的用量额度没有用完。

Q2 需要花很多时间吗?

需要。这项工作本身具有连续性，时间主要花在一轮一轮的确认和验收上。

Q3 可以在实际项目中使用吗?

可以，条件是：人能定义目标、能验收结果、数据允许进入所用模型。涉密数据与无法验收的生产系统，不要交给 AI。

核心结论

“人工智能目前可以完成一个复杂工程。”

人负责目标、取舍和验收，AI 负责执行。

AI生成

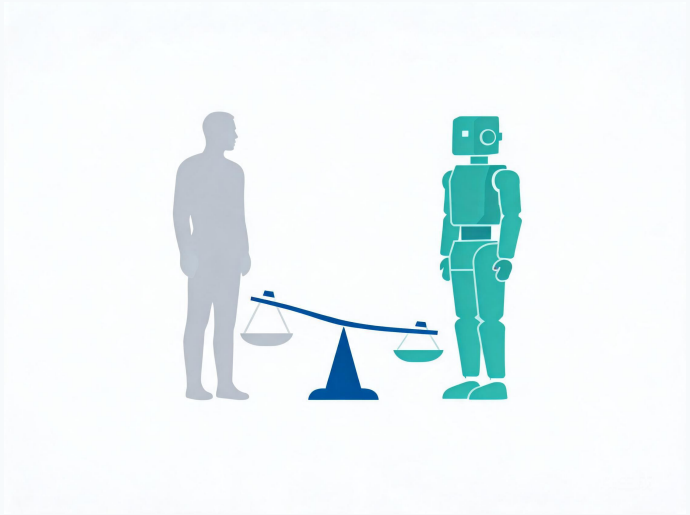
100% AI Coding（尽最大可能让AI来完成全部工作）

AI：执行者与建议者

代码、方案、文案、软著草稿、推广稿，尽最大可能由 AI 撰写——使用 AI 越充分，最终收益越大。

人：产品负责人与验收者

- 确定目标、做取舍、验收结果
- 申请开发者账号，处理协议与税务
- 真机测试与审核回复
- 提交软件著作权 · 推广投稿



“人负责把要求说清楚，并做出判断。”

和AI携手沟通解决问题

- **人规划，AI 执行**
- **人决定，AI 遵守**
- **人提问，AI 回答**
- **AI 提出建议，人做决策**
- **AI 列举选项，人做取舍**

每一轮：先说明要什么 → AI 出方案 → 人确认后再动手



人提要求，AI 干活；AI 提建议，人做决定

四个重要技巧

善用rules，提前为AI制定好规则

编码规范、界面风格、兼容性与文风，提前写进每次对话 AI 都能看见的 Rules；功能完成后再补，上架前会集中返工。

准确的表达、明确的指令，提高代码有效性

页面如何划分、点击何处出现什么、哪些字段必填，都写到能够验收；具体实现交给 AI。

张弛有度，适时让AI自己发挥，会有额外惊喜

总体方向由人把握；尚未想清楚的交互，让 AI 先给几种做法并比较优缺点，再由人选择。

让AI来查AI（让英雄来查英雄，让好汉来查好汉）

一段功能完成后复查；收尾与上架前收紧任务，只查找重大缺陷，不再增加新功能。

我踩的坑及相应的对策：八条实战经验

1. **你是产品经理，不是程序员**
   下达可验收的指令，不介入具体实现

2. **要多想但也别想过多**
   方向正确即可推进，不被单页细节挡住

3. **商店规则会改写商业设想**
   买断改为免费下载 + 内购，提前确认规则

4. **人教 AI 教不会，图教 AI 一把就会了**
   界面问题直接截图标注，AI 更容易定位

5. **反复修不好，不如重写**
   绕进错误路径时，重写比继续修补更有效

6. **AI 说完成了，验收仍然由人来做**
   AI 说做完了，只说明它认为做完了

7. **不要重复造轮子**
   复用现成能力与已有界面

8. **模型千千万，各有各的优点**
   按任务选择模型

1.引入标准开发流程：每一轮都走同一条路
2.just do it：实际走一遍全流程，比什么都管用
3.备份是个好习惯：没事咱就commit



为什么需要标准开发流程

隔几天再打开工程，人很容易忘记上一轮做到哪里。每一轮只做一个页面或一件事，走同一个循环，就不靠记忆推进。

上架、内购等隔一段时间还会再用的决定，写成文档保留。

休息一下

玩个互动小游戏吧

放松两分钟，回来进入 06 · 实操时间线。

AI生成

三个月，三段推进

阶段一

8.04 — 9.21

功能按页面循环推进

一轮只做一个页面或一件事，设计与编码交替，测试穿插其中

主界面 → 头像昵称 → 图标 → 录入页 → 查询排错 → 设置页 → 功能复查

阶段二

9.21 — 9.30

上架前集中处理

开发者账号、内购、隐私政策、兼容性与验收，集中补齐

兼容补齐 → 手工测试 → 账号与内购 → 正式上架

阶段三

10.07 — 10.12

上架后并行推进

根据第一批用户反馈发布新版本，并行推进推广与软著

v1.0.2 → 软著申请 → 推广投稿

用数据说话：103 小时、60 美元

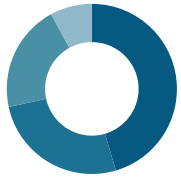
103 h	88 次	30.7%	24 个	$60
Cursor 协作总时长	**功能审核确认**	**返工率**	**Bug**	**两个月模型费用**
功能 69.6 · 上架 26.1 · 文档 5.2	先确认方案，再开始开发	27 次返工 ÷ 88 次确认	行为与既定规则不符	高强度使用，额度未用完

103 小时的主要构成



- 功能开发　69.6 h
- 上架　26.1 h
- 文档　5.2 h

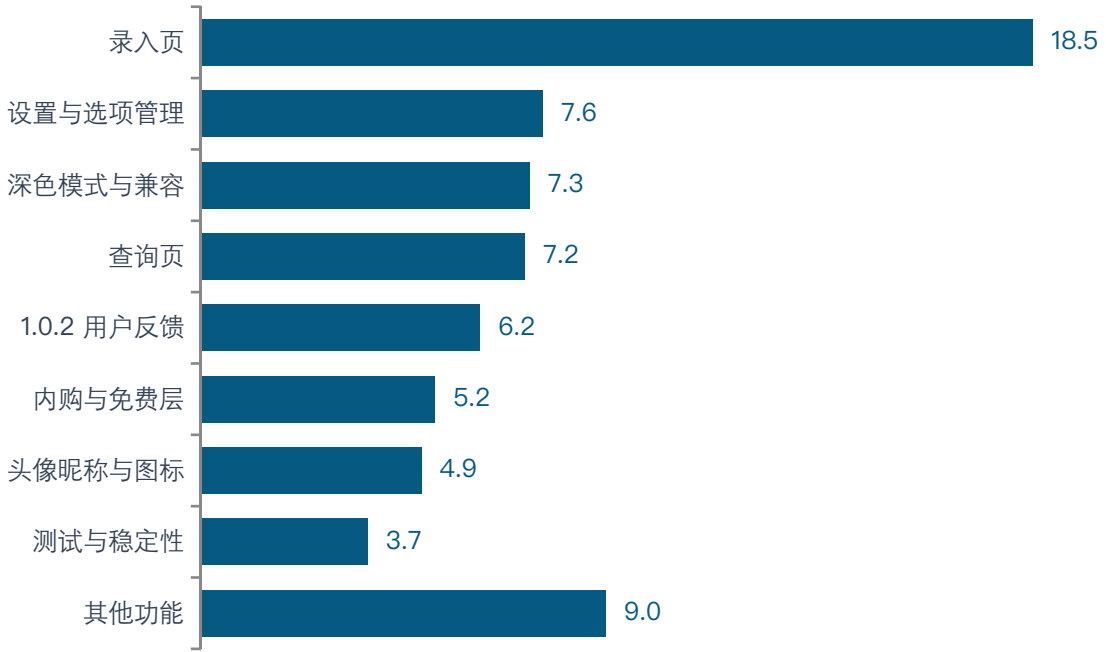
88 次功能审核确认的构成



- 新功能　40 次 · 45%
- 返工　23 次 · 26%
- 缺陷修复　18 次 · 20%
- 仅复查　7 次 · 8%

14 项功能中 9 项发生过返工——返工是 AI Coding 的常态，人的验收能力决定效率。

功能开发工时分布（合计 69.6 小时）



解读

- 录入页开发耗时最多：交互密度最高的页面最耗时
- 查询页返工次数最多：查询逻辑越复杂越容易出错
- 设置页出现了唯一需要重写的恶性bug：牵扯面越广牵扯程度越深的工作最容易出严重bug
- 兼容性占 10.5%：若从第一天写入 Rules，可省下这笔返工

上架对话 26.1 小时、软著与推广文档 5.2 小时另计，三部分合计约 103 小时。

结语

瓶颈在想法是否值得做，
难点在协作是否有效，
风险在隐私能否守住。

—— 与每一位想尝试 AI Coding 的人共勉

AI生成

自由交流 · 谢谢观看

把目标说清楚，把验收做到位，剩下的交给 AI

邮箱：fingalyang@outlook.com

App Store：https://apps.apple.com/cn/app/份子钱银行/id6817363332

GitHub：https://github.com/xray1224/AI-Coding-Guide-IOS-APP

Gitee：https://gitee.com/fingalyang/AI-Coding-Guide-IOS-APP

感谢陈老师开发这门课程，感谢李老师、任老师的专业工作

AI生成